"Ss. Cyril and Methodius" University in Skopje
FACULTY OF COMPUTER
SCIENCE AND ENGINEERING

Faculty of computer science and engineering

Asynchronous JavaScript and DOM Applications

Advanced Web Design

Asynchronous JavaScript

- Network requests and timers finish after the current code continues.
- A callback handles work that should run later.
- The page should remain usable while data is loading.

Promises

- A Promise represents an operation that has not necessarily finished.
- Its state is pending, fulfilled or rejected.
- A settled Promise keeps its result.

Creating a Promise

- Example: waiting without stopping the rest of the script.

```
function orderPizza(pizza) {
    return new Promise(resolve => {
        // Simulate the time it takes to make the pizza.
        setTimeout(() => {
            resolve(`Your ${pizza} pizza is ready for pickup!`);
        }, 3000);
    });
}

const order = orderPizza("Margherita");

order.then(message => {
    console.log(message);
});

console.log("Order placed. You can keep browsing the menu.");
```

then(), catch() and finally()

- then() handles a fulfilled value and returns a new Promise.
- catch() handles a rejection.
- finally() performs cleanup after success or failure.

Returning values in a chain

- Example: return the result that the next callback needs.

```
const pricePerPizza = 350;

Promise.resolve(2) // The quantity is already known.
    .then(quantity => {
        return quantity * pricePerPizza;
    })
    .then(total => {
        console.log(`Order total: ${total} MKD`);
    })
    .catch(error => console.error(error.message))
    .finally(() => console.log("Finished calculating the total."));
```

async / await

- An async function always returns a Promise.
- await pauses that async function until the result is available.
- Waiting does not stop other JavaScript from continuing.

async / await

- Example: an async function can return a normal value.

```
async function getNumber() {
    return 42;
}

console.log(getNumber()); // Promise
getNumber().then(number => console.log(number)); // 42

async function run() {
    const number = await getNumber();
    console.log(number); // 42
}
run();
```

HTTP requests and JSON

- GET requests retrieve data from an API endpoint.
- A response contains a status code and a body.
- JSON is text that must be parsed into JavaScript data.

fetch() in two steps

- First await the response, then await the parsed JSON body.

```
async function loadUsers() {
    const response = await fetch(
        "https://jsonplaceholder.typicode.com/users");
    if (!response.ok) {
        throw new Error(`HTTP ${response.status}`);
    }
    const users = await response.json();
    return users;
}

loadUsers().then(console.table).catch(console.error);
```

Handling errors

- fetch() rejects on network failures.
- HTTP 404 or 500 responses require an explicit response.ok check.
- response.json() can reject when the response body is invalid JSON.

Parallel requests

- Start independent requests together using Promise.all().
- Results keep the order of the input Promises.
- If one Promise rejects, Promise.all() rejects.

```
async function loadDashboard() {
    const base = "https://jsonplaceholder.typicode.com";

    const [usersResponse, postsResponse] = await Promise.all([
        fetch(`${base}/users`),
        fetch(`${base}/posts`)
    ]);

    if (!usersResponse.ok || !postsResponse.ok) {
        throw new Error("Could not load dashboard data.");
    }

    const [users, posts] = await Promise.all([
        usersResponse.json(),
        postsResponse.json()
    ]);

    return { users, posts };
}
```

Async callbacks and arrays

- forEach() does not wait for async callbacks.
- Use map() and Promise.all() when all results are needed.

```
async function loadUsersById(ids) {
    const requests = ids.map(async id => {
        const response = await fetch(
            `https://jsonplaceholder.typicode.com/users/${id}`
        );

        if (!response.ok) {
            throw new Error(`Could not load user ${id} (HTTP $
{response.status})`);
        }

        return response.json();
    });

    return Promise.all(requests);
}

loadUsersById([1, 2, 3])
    .then(users => console.table(users))
    .catch(error => console.error(error.message));
```

API data in the DOM

- Keep the request, data transformation and rendering separate.
- Display a loading state, success, an empty result or an error.

Example 1

- Write a function that fetches users and posts from https://jsonplaceholder.typicode.com/users and https://jsonplaceholder.typicode.com/posts. Store each user's id, name, username and email, and each post's userId, title and body, using plain objects. Write a summary function that returns the first 30 characters of a post's body followed by ... if it is longer. After fetching the data, filter posts to keep only those whose titles are at least 20 characters long, check whether some posts contain "qui" in either the title or body and whether every user has at least one post, map the authors into readable strings that include the user's name and post count, and sort those authors by post count in descending order. Print all results in a clean, readable format and handle invalid payloads, empty arrays and request errors.

Solution

```
function summarizeBody(body){
    return body.length > 30 ? body.slice(0,30) + '...' : body
}

async function fetchData() {
    let usersResponse = await fetch("https://jsonplaceholder.typicode.com/users")
    let usersData = await usersResponse.json()

    let postsResponse = await fetch("https://jsonplaceholder.typicode.com/posts")
    let postsData = await postsResponse.json()

    // Store each user's id, name, username and email,
    // and each post's userId, title and body, using plain objects.

    usersData = usersData.map(user => ({
        id: user.id,
        name: user.name,
        username: user.username,
        email:user.email
    }))
    console.log('users', usersData)
```

Solution

```
postsData = postsData.map(post => ({
    userId: post.userId,
    title: post.title,
    body: summarizeBody(post.body)
})).filter(post => post.title.length >= 20)

let postContainsQui = postsData.some(post => post.title.toLowerCase().includes("qui")
    || post.body.toLowerCase().includes("qui"))

let userIdsWithPostsList = postsData.map(post => post.userId)

let allUsersHavePosts = usersData.every(user => userIdsWithPostsList.includes(user.id))

console.log(`Posts contain qui: ${postContainsQui}`)
console.log(`All users have posts: ${allUsersHavePosts}`)
```

Solution

```
let usersDataWithPostsCount = usersData
        .map(user => ({...user, postsCount: postsData.filter(post => post.userId ===
user.id).length}))
        .sort((a,b) => b.postsCount - a.postsCount)
        .map(user => `${user.name} - ${user.postsCount} posts`)

    console.log('usersDataWithPostsCount', usersDataWithPostsCount)

}

fetchData()
```

Example 2

- Write an asynchronous JavaScript function that fetches the first 20 characters from https://rickandmortyapi.com/api/character (GET request) and then fetches details for each character's first episode. After fetching the data, filter to keep only Alive characters, extract the unique set of species and print them alphabetically, sort the full list by episodeCount in descending order and show the top five characters by appearances. Check whether some characters have an origin that includes "Earth" and finally print everything in a clean, readable format.

Solution

```
async function fetchData() {
    let response = await fetch('https://rickandmortyapi.com/api/character')
    let charactersData = await response.json()
    let characters = charactersData.results

    // fetch all first episodes
    let episodesResponses = await Promise.all(characters.map(async character => await
fetch(character.episode[0])))
    let episodes = await Promise.all(episodesResponses.map(async response => await
response.json()))

    characters.forEach((character, index) => {
        character.firstEpisode = episodes[index]
        character.episodeCount = character.episode.length
    })
    console.log('characters', characters)

    let aliveCharacters = characters.filter(character => character.status == 'Alive')
    console.log('Alive characters: ', aliveCharacters)
```

Solution

```
  let species = new Set(characters.map(character => character.species).sort())
     console.log("Species:", species)

     let topFiveCharactersByAppearance = characters.sort((a,b) => b.episodeCount-
a.episodeCount).slice(0,5)
     console.log("Top five characters by appearance:", topFiveCharactersByAppearance)

     let charactersFromEarth = characters.filter(character =>
character.origin.name.includes('Earth'))
     console.log('Characters from Earth: ', charactersFromEarth)
 }

fetchData()
```

Example 3

- Create a page with a "Load users" button, a status message and a list. When the button is clicked, fetch users from https://jsonplaceholder.typicode.com/users and display their names alphabetically. Disable the button while loading and show a loading message. If the returned array is empty, display "No users found".

Solution

```
async function fetchAndDisplayUsers(){
    let users = fetch('https://jsonplaceholder.typicode.com/users')
        .then(response => {
            document.getElementById("loading-message").innerHTML = "Loading..."
            document.getElementById("fetch-btn").disabled = true
            return response.json()
        })
        .then(users => {
            let mainSectionElement = document.getElementById("main-section")
            mainSectionElement.innerHTML = ""
            if(users.length > 0){
                users.sort((a,b) => a-b)
                    .forEach(user => {
                        let userElement = document.createElement("h3")
                        userElement.innerHTML = user.name
                        mainSectionElement.appendChild(userElement)
                    })
            }
```

Solution

```
else{
                let emptyArrayMessageElement = document.createElement("h3")
                emptyArrayMessageElement.innerHTML = "No users found"
                mainSectionElement.appendChild(emptyArrayMessageElement)
            }

        })
        .catch(error => console.error(error))
        .finally(() => {
            document.getElementById("loading-message").innerHTML = ""
            document.getElementById("fetch-btn").disabled = false
        })
}
```

Example 4

- Fetch data for products from https://dummyjson.com/products (GET request) and perform useful data transformations: extract all existing categories and print them alphabetically. Print out the three products with the highest average review rating and the three with the lowest. If a product has no reviews, treat its average as 0. Build a summary per category containing the number of products, the total inventory value (price * stock summed). Print all products that are low stock (stock < 10) sorted by stock ascending. Print the top 5 discounted products by discount percentage (descending) showing Title - Price - Discount%. Finally, produce a quick availability report counting products by availability status, print everything in a readable format and handle missing/empty arrays appropriately.

Solution

```
async function fetchData(){
    let response = await fetch('https://dummyjson.com/products')
    let jsonData = await response.json()
    products = jsonData.products

    let categories = new Set(products.map(product => product.category))
    console.log('Categories: ', categories)

    products.forEach(product => {
        let reviewsSum = product.reviews.reduce((sum, review) => review.rating+sum, 0)
        product.averageRating = reviewsSum / product.reviews.length
    })

    products = products.sort((a,b) => a.averageRating - b.averageRating)

    console.log("Top 3 products with highest average rating: ")
    products.slice(products.length-3).forEach(product => console.log(product.title))

    console.log("Top 3 products with lowest average rating: ")
    products.slice(0, 3).forEach(product => console.log(product.title))
```

Solution

```
let summaryPerCategory = []
categories.forEach(category => {
    let productsforCategory = products.filter(product => product.category == category)
    let numProducts = productsforCategory.length
    let totalInventoryValue = productsforCategory.reduce((sum, product) => sum +
(product.stock + product.price), 0)

    summaryPerCategory.push({
        category,
        numProducts,
        totalInventoryValue
    })
})

console.log('Summary per category', summaryPerCategory)

let lowStockProducts = products.filter(product => product.stock < 10)
    .sort((a,b) => a.stock - b.stock)

console.log("Low stock products: ", lowStockProducts)
```

Solution

```
console.log("Top 5 discounted products by discount percentage (descending):")
    products.sort((a,b) => b.discountPercentage-a.discountPercentage).slice(0,5)
        .forEach(product => console.log(`${product.title} - ${product.price} - $
{product.discountPercentage}%`))


    let availabilityStatusReportMap = {}

    products.forEach(product => {
        if(product.availabilityStatus in availabilityStatusReportMap){
            availabilityStatusReportMap[product.availabilityStatus] += 1
        }
        else{
            availabilityStatusReportMap[product.availabilityStatus] = 1
        }
    })

    console.log("Availability status report: ", availabilityStatusReportMap)
}

fetchData()
```